

**Tribhuvan University**  
**Faculty of Humanities and Social Studies**

## **Diabetes Prediction System**

### **A PROJECT REPORT**

**Submitted to:**  
**Department of Computer Application**

**Prime College**

***In the partial fulfillment of the requirements for the Bachelors in Computer  
Application***

**Submitted by:**  
**Piyush Sthapit (41002140)**  
**April 2026 A.D**

**Under the supervision of**  
**Mr. Sravan Ghimire**



**Tribhuvan University**

**Faculty of Humanities and Social Science**

**Prime College**

**Supervisor's recommendation**

I hereby recommend that this project is prepared under my supervision by Piyush Sthapit entitled “**Diabetes Prediction System**” in the partial fulfillment for the degree of Bachelor of Computer Application is recommended for the final evaluation.

Mr. Sravan Ghimire

**SUPERVISOR**

**Prime College**



**Tribhuvan University**

**Faculty of Humanities and Social Science**

**Prime College**

## **LETTER OF APPROVAL**

This is to certify that this project is prepared by Piyush Sthapit entitled “**Diabetes Prediction System**” in the partial fulfillment for the degree of Bachelor of Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

|                                                                                 |                                                                                                                                     |
|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mr. Sravan Ghimire</b><br><b>Supervisor</b><br><b>Prime College, Khusibu</b> | <b>Er. Rolisha Sthapit</b><br><b>BCA Co-Ordinator/ Internal Examiner</b><br><b>BCA Department</b><br><b>Prime College, Khusibun</b> |
|                                                                                 | <b>Mr. Kumar Prasun</b><br><b>External Examiner</b>                                                                                 |

## ABSTRACT

This project is focused on predicting diabetes risk using patient health metrics, aiming to classify individuals into high-risk or low-risk categories for developing diabetes. The system is designed to provide personalized health insights, early warning assessments, and tailored medical recommendations. The project is implemented using Python, leveraging machine learning techniques for medical classification. The patient data is preprocessed through cleaning and transformation, handling outliers, and converting health metrics into optimized feature representations using methods like normalization and feature engineering. The primary machine learning algorithm, K-Nearest Neighbors (KNN) with hyperparameter tuning, is complemented by other models to train and evaluate the prediction system. The models are assessed using performance metrics including accuracy, precision, recall, and F1-score to ensure reliable diabetes risk prediction. The system provides a simple and efficient approach to analyze individual health profiles, enabling users to quickly understand their diabetes risk level and receive appropriate medical guidance. By automating diabetes risk assessment, this project reduces manual screening effort and facilitates data-driven healthcare decisions for applications like preventive medicine, personalized treatment planning, and early intervention strategies. Overall, the project demonstrates how machine learning techniques can be effectively applied to health data for extracting meaningful clinical insights and improving diabetes prevention outcomes.

**Keywords:** *diabetes prediction, health analytics, K-Nearest Neighbors, medical classification, machine learning, Python, risk assessment, healthcare informatics, predictive modeling*

## ACKNOWLEDGMENT

For the partial fulfillment of the project in this semester, I would like to express my sincere gratitude to everyone who had directly and indirectly helped me to complete this project. There were times when I thought that this project was too difficult or I may not be able to complete this in time, but I'm very thankful to everyone who had encouraged, inspired, and helped me to complete this project. I am thankful to our supervisor **Mr. Sravan Ghimire** and our coordinator **Er. Rolisha Sthapit** who helped me in each difficulty that aroused during this project.

I would also like to express my heartfelt gratitude to all the classmates who were always inspiring me to continue with the project and reminded me of the competition every time. I am also grateful to all our professors for their guidance and support throughout the project.

Finally, I am grateful to Tribhuvan University for giving this opportunity via the course of Computer Application to help me understand the project ethics at this early stage and helped me develop my skills and knowledge through this project.

Thank you all for your invaluable contributions.

Sincerely,

Piyush Sthapit

# TABLE OF CONTENTS

|                                                               |             |
|---------------------------------------------------------------|-------------|
| <b>Supervisor’s recommendation.....</b>                       | <b>i</b>    |
| <b>LETTER OF APPROVAL.....</b>                                | <b>ii</b>   |
| <b>TABLE OF CONTENTS .....</b>                                | <b>v</b>    |
| <b>LIST OF TABLES.....</b>                                    | <b>viii</b> |
| <b>LIST OF FIGURES .....</b>                                  | <b>ix</b>   |
| <b>CHAPTER 1 INTRODUCTION .....</b>                           | <b>1</b>    |
| 1.1 Introduction .....                                        | 1           |
| 1.2 Problem Statement .....                                   | 2           |
| 1.3 Objectives.....                                           | 2           |
| 1.4 Scopes and Limitations.....                               | 3           |
| 1.4.1 Scopes .....                                            | 3           |
| 1.4.2 Limitations .....                                       | 3           |
| 1.5 Development Methodology Diabetes Prediction System .....  | 3           |
| 1.6 Report Organization.....                                  | 5           |
| <b>CHAPTER 2 BACKGROUND STUDY AND LITERATURE REVIEW .....</b> | <b>7</b>    |
| 2.1 Background Study.....                                     | 7           |
| 2.2 Literature Review.....                                    | 8           |
| <b>CHAPTER 3 SYSTEM ANALYSIS AND DESIGN.....</b>              | <b>12</b>   |
| 3.1 System Analysis.....                                      | 12          |
| 3.1.1 Requirement Analysis.....                               | 12          |
| i. Functional Requirements.....                               | 13          |
| ii. Non-Functional Requirements.....                          | 15          |
| 3.1.2 Feasibility Study.....                                  | 16          |
| 3.1.3 Object Modeling: Object and class Diagram.....          | 19          |
| 3.1.4 Dynamic Modeling: State and Sequence Modeling .....     | 19          |
| 3.1.3 Process Modeling: Activity Diagram .....                | 22          |
| 3.2 System Design .....                                       | 23          |
| 3.2.2 Component Diagram .....                                 | 25          |
| 3.2.3 Deployment Design.....                                  | 26          |
| 3.3 Algorithm Details .....                                   | 26          |
| <b>CHAPTER 4 IMPLEMENTATION AND TESTING.....</b>              | <b>30</b>   |
| 4.1 Implementation.....                                       | 30          |
| 4.1.1 Front End Tools .....                                   | 30          |
| 4.1.2 Backend Tools .....                                     | 30          |
| 4.1.3 Implementation details of modules.....                  | 30          |
| 4.2 Testing .....                                             | 36          |
| 4.2.1 Unit Testing .....                                      | 36          |

|                                                              |           |
|--------------------------------------------------------------|-----------|
| 4.3 Result Analysis .....                                    | 41        |
| <b>CHAPTER 5 CONCLUSION AND FUTURE RECOMMENDATIONS .....</b> | <b>45</b> |
| 5.1 Lessons Learned .....                                    | 45        |
| 5.2 Conclusion .....                                         | 45        |
| 5.3 Future Recommendations .....                             | 46        |
| <b>REFERENCE .....</b>                                       | <b>47</b> |
| <b>APPENDICES</b>                                            |           |

## **LIST OF ABBREVIATIONS**

AI: Artificial Intelligence

BMI: Body Mass Index

CSS: Cascading Style Sheet

CSV: Comma-Separated Values

FINDRISC: Finnish Diabetes Risk Score

HTML: Hypertext Markup Language

JSON: JavaScript Object Notation (Data Storage Format)

KNN: K-Nearest Neighbors

ML: Machine Learning

PHP: Preprocess Hypertext Preprocessor

WHO: World Health Organization

## LIST OF TABLES

|                                 |    |
|---------------------------------|----|
| Table 3.1: Gantt Table .....    | 18 |
| Table 4.1: Unit Testing.....    | 37 |
| Table 4.2: User Interface ..... | 39 |
| Table 4.3: Admin Interface..... | 40 |

## LIST OF FIGURES

|                                                                                  |    |
|----------------------------------------------------------------------------------|----|
| Figure 1.1: Development Methodology of Diabetes Prediction System.....           | 5  |
| Figure 3.1: Use case diagram of Diabetes Prediction System.....                  | 14 |
| Figure 3.2: Gantt Chart of Diabetes Prediction System .....                      | 18 |
| Figure 3.3: Class diagram of Diabetes Prediction System.....                     | 19 |
| Figure 3.4: State Diagram of Diabetes Prediction.....                            | 20 |
| Figure 3.5: Sequence diagram of Diabetes Prediction System.....                  | 21 |
| Figure 3.6: Activity diagram of Diabetes Prediction System .....                 | 22 |
| Figure 3.7: System Architecture of Diabetes Prediction System.....               | 24 |
| Figure 3.8: Component Diagram.....                                               | 25 |
| Figure 3.9: Deployment Design of Diabetes Prediction System.....                 | 26 |
| Figure 4.1: Login Module .....                                                   | 31 |
| Figure 4.2: Scan Module .....                                                    | 32 |
| Figure 4.3: Diabetes Prediction Module .....                                     | 33 |
| Figure 4.4: Code of KNN Algorithm .....                                          | 34 |
| Figure 4.5: Code for Generating Confusion Matrix and Classification Report... .. | 35 |
| Figure 4.6: Classification Report and Confusion Matrix .....                     | 43 |
| Figure 4.7: Loss vs Accuracy Curve.....                                          | 44 |

# CHAPTER 1

## INTRODUCTION

### 1.1 Introduction

Diabetes prediction through health analytics, also known as medical risk assessment, is the process of identifying and classifying individuals' likelihood of developing diabetes based on health parameters, and it has become increasingly important with the growth of digital health data. Patients generate various health metrics through medical examinations, wearables, and health apps, creating large volumes of data that are difficult to manually analyze and interpret comprehensively. Patient health profiles, consisting of metrics like glucose levels, BMI, age, and other physiological measurements, are particularly useful for diabetes prediction because they often reflect underlying health patterns and risk factors. However, health data is also challenging to analyze because it can contain variations, outliers, complex interactions between variables, and measurement inconsistencies.

Analyzing the diabetes risk from health metrics can provide valuable insights for healthcare providers, patients, public health officials, and medical researchers. For instance, healthcare providers can identify high-risk patients for early intervention, patients can receive personalized health recommendations, public health organizations can track population health trends, and researchers can identify risk patterns for targeted prevention strategies. Due to the large volumes of health data generated in medical settings, automated methods using machine learning have become essential for efficient diabetes risk assessment. Machine learning algorithms, such as K-Nearest Neighbors (KNN) with hyperparameter optimization, can be trained on medical datasets to learn patterns in health metrics and classify new patients as high-risk or low-risk for developing diabetes.

Python, with its powerful libraries like Scikit-learn, Pandas, and Matplotlib, provides the tools required for data preprocessing, feature engineering, model training, and performance evaluation. This project focuses on performing diabetes prediction using patient health data by cleaning the medical records, transforming health metrics into

predictive features, and building models to classify diabetes risk accurately. By automating this process, the project demonstrates how machine learning can be applied to healthcare data to understand individual health risks efficiently, detect early warning signs, and support informed clinical decision-making in areas such as preventive medicine, personalized healthcare, and population health management.

## **1.2 Problem Statement**

Diabetes is a growing global health concern, and early identification of individuals at risk is essential for effective prevention and treatment. However, manually assessing diabetes risk using individual health metrics such as glucose level, BMI, age, and other medical parameters is time-consuming and inefficient, especially with the increasing volume of patient data in healthcare systems. Healthcare professionals must review multiple indicators for each patient, which can lead to delays and inconsistent evaluations.

Traditional health risk assessment methods, including manual screening and simple rule-based scoring systems, often fail to capture the complex relationships and non-linear patterns that exist among health variables. Variations in measurement units, missing values, outliers, and subtle interactions between different health factors make accurate risk prediction challenging without advanced analytical techniques. As a result, patients may receive delayed or less precise risk assessments.

There is a need for an automated and intelligent system that can efficiently analyze large volumes of health data and provide accurate, consistent, and objective diabetes risk predictions. A machine learning-based diabetes prediction system addresses this problem by automatically classifying individuals into high-risk or low-risk categories based on their health metrics. Such a system reduces clinical workload, improves screening efficiency, and supports early intervention and informed healthcare decision-making.

### **1.3 Objectives**

- To classify diabetes risk assessment using machine learning to classify patients into high-risk and low-risk categories for timely intervention.
- To provide an accessible and user-friendly digital platform that enables individuals to assess their diabetes risk early, encouraging timely medical consultation and promoting proactive health management.

### **1.4 Scopes and Limitations**

#### **1.4.1 Scopes**

The project aims to simplify and automate diabetes risk prediction using patient health data. It involves preprocessing medical metrics to handle outliers and missing values, and converting health parameters into numerical features suitable for machine learning. A K-Nearest Neighbors (KNN) model with hyperparameter tuning is implemented to classify patients as high-risk or low-risk for diabetes. The project provides insights into individual health risks and population trends, helping healthcare providers, patients, and public health planners make informed decisions. The scope is limited to health-data-based risk classification using supervised machine learning techniques.

#### **1.4.2 Limitations**

- The system focuses only on health-data-based risk classification of patient metrics and does not handle genomic data, medical imaging, or real-time physiological monitoring.
- Accuracy may be affected by incomplete health records, measurement variability, or missing values, and the project does not include advanced techniques like deep neural networks or real-time clinical data integration.

### **1.5 Development Methodology Diabetes Prediction System**

The diabetes prediction project was developed using an iterative methodology, following a structured and systematic approach to ensure effective planning, implementation, and validation. The development process involved the following steps:

### **i. Requirement Gathering**

In the first phase, the requirements for the system were gathered by understanding the need to analyze diabetes risk from patient health data. Discussions and research were conducted to identify key objectives, such as classifying patients into high-risk and low-risk categories, preprocessing health metrics, and evaluating model performance. This phase focused on defining the scope and functionality clearly before development.

### **ii. System Design**

Based on the gathered requirements, the system design was planned. The workflow of data preprocessing, feature engineering, model training, and evaluation was structured carefully. Decisions on using the K-Nearest Neighbors (KNN) algorithm with hyperparameter tuning, as well as Python libraries like Scikit-learn, Pandas, and Matplotlib, were made. This phase ensured a logical flow and a clear framework for implementation.

### **iii. Implementation (Development)**

In this phase, the data preprocessing scripts were developed to clean and transform health metrics into standardized numerical features using normalization and feature selection. The machine learning model was implemented using Python, and the training and testing pipeline was created. The model was trained on labeled health data and optimized to achieve reliable diabetes risk classification.

### **iv. Testing**

After implementation, the model was evaluated using metrics such as accuracy, precision, recall, and F1-score. Testing included validating preprocessing steps, feature engineering, and model performance to ensure the system classified diabetes risk correctly and consistently.

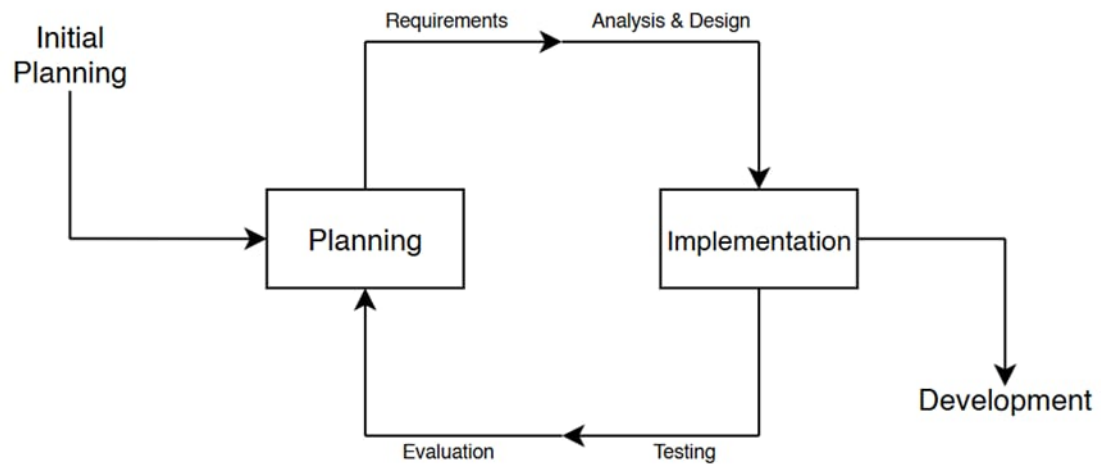
### **v. Deployment**

Once testing was completed, the system was prepared for deployment. The final scripts and model were organized for easy execution, allowing users to input health metrics and receive risk classification results efficiently.

### **vi. Maintenance**

The system is designed to allow updates, such as retraining the model with new health

data or improving preprocessing techniques. Regular monitoring and updates ensure the system remains accurate and relevant for ongoing clinical and preventive healthcare use.



**Figure 1.1: Development Methodology of Diabetes Prediction System**

Here’s a visual representation of the **Development Methodology** for The Diabetes Prediction System. The flowchart shows the key stages in the development process, with arrows indicating the progression from one phase to the next.

## 1.6 Report Organization

This report is organized into five chapters to comprehensively document the development of the diabetes prediction system using patient health data.

**Chapter 1: Introduction** provides an overview of the project, including the background, problem statement, objectives, development methodology, scope, limitations, and significance. It establishes the importance of automating diabetes risk assessment and outlines the purpose of the project.

**Chapter 2: Literature Review** explores existing research, systems, and techniques related to diabetes prediction and machine learning in healthcare. This chapter discusses relevant methodologies, algorithms, and tools to ensure the project aligns with best practices and achieves reliable clinical performance.

**Chapter 3: System Design and Methodology** details the process of data collection, preprocessing, feature engineering, and model implementation. It also explains the

workflow, architecture, and tools used to develop the system, providing diagrams and explanations to illustrate the methodology clearly.

**Chapter 4: Implementation and Testing** describe the development process using Python, the machine learning models applied, and the testing methods used to evaluate model performance. It includes the results of the experiments, validation of the K-Nearest Neighbors (KNN) model with hyperparameter tuning, and assessment of the system's accuracy and reliability in clinical contexts.

**Chapter 5: Conclusion and Future Recommendations** summarize the key findings and outcomes of the project, highlighting its contributions and limitations. It also provides recommendations for improving the system and extending its functionality for future research and clinical applications.

## **CHAPTER 2**

### **BACKGROUND STUDY AND LITERATURE REVIEW**

#### **2.1 Background Study**

Diabetes risk prediction is the process of assessing and categorizing an individual's likelihood of developing diabetes based on health parameters such as glucose levels, BMI, age, and related physiological indicators. With advances in digital health records and wearable devices, large volumes of patient health data are generated daily, often including structured medical metrics, demographic information, and lifestyle factors. This makes manual screening and risk stratification both challenging and inefficient.

Traditional methods of diabetes risk assessment, such as manual clinical evaluation or rule-based scoring systems like the FINDRISC questionnaire, present several limitations:

- Time-consuming and inefficient for analyzing large patient populations in healthcare systems.
- Prone to subjective interpretation and inconsistency across different healthcare providers.
- Difficulty in capturing complex, non-linear interactions among multiple health variables.
- Challenges in scaling and updating risk evaluation methods as new data becomes available or guidelines evolve

With the rapid advancement of healthcare technology and data science, machine learning has emerged as a powerful tool for medical risk prediction. Automated diabetes risk prediction systems have been developed to address the increasing demand for efficient and accurate analysis of patient health data. These systems utilize machine learning algorithms to process large volumes of medical information and classify individuals based on their likelihood of developing diabetes.

Python has become one of the most widely used programming languages in healthcare data analysis due to its simplicity and extensive ecosystem of libraries. Tools such as Scikit-learn, Pandas, and NumPy are commonly used for data preprocessing, feature engineering, and model training. In such systems, preprocessing plays a crucial role and typically involves handling missing values, normalizing data, and transforming health metrics into standardized numerical formats. Techniques such as feature scaling and standardization help improve model accuracy and ensure consistent performance.

By using machine learning models such as K-Nearest Neighbors with hyperparameter tuning, diabetes prediction systems can classify patient risk accurately and consistently, reducing manual screening efforts and improving efficiency in processing extensive clinical datasets.

## **2.2 Literature Review**

### **[1]: Traditional Methods Limitations**

**Introduction:** Traditional approaches to diabetes risk assessment have relied heavily on manual evaluation methods that present significant challenges in modern healthcare settings. Clinical rule-based scoring systems and manual chart reviews have been the standard practice for decades, but their effectiveness is increasingly questioned as healthcare data volumes grow exponentially.

**Objective:** To examine the limitations of conventional diabetes screening methods and establish the need for automated approaches that can address the inherent inefficiencies of manual assessment.

**Method:** The literature was reviewed focusing on studies that evaluated traditional diabetes screening tools such as the FINDRISC questionnaire, ADA risk assessment tools, and manual clinical chart reviews. Comparative analyses examined time requirements, consistency across practitioners, and scalability limitations.

**Conclusion:** Traditional methods are demonstrably time-consuming, prone to inter-practitioner variability, and fundamentally inefficient for population-level screening. These limitations establish the clear need for automated systems that can process large volumes of patient data consistently and efficiently.

## **[2]: Automated Systems Benefits**

**Introduction:** The emergence of automated diabetes prediction systems represents a paradigm shift in how healthcare providers approach risk assessment and population health management. These systems leverage computational power to overcome the limitations inherent in manual approaches.

**Objective:** To evaluate the demonstrated benefits of automated diabetes prediction systems in clinical and research settings, particularly regarding reliability, consistency, and efficiency metrics.

**Method:** Studies implementing automated prediction systems were analyzed for performance metrics including processing speed, classification consistency, and correlation with clinical outcomes. Comparisons were drawn between automated and manual approaches using standardized datasets.

**Conclusion:** Automated diabetes prediction systems provide reliable risk classification while substantially reducing clinical workload. These systems demonstrate superior consistency compared to human assessment and enable large-scale screening that would be impractical with manual methods alone.

## **[3]: SVM Applications**

**Introduction:** Support Vector Machines have emerged as a powerful classification technique in healthcare applications, offering robust performance on medium-dimensional datasets with clear margin separation. Their application to diabetes prediction has been extensively studied due to their theoretical strengths in handling complex decision boundaries.

**Objective:** To assess the effectiveness of SVM algorithms in diabetes risk prediction tasks and identify optimal configuration parameters for healthcare applications.

**Method:** Studies employing SVM for diabetes prediction were reviewed, with particular attention to kernel function selection, hyperparameter optimization approaches, and reported accuracy metrics on standard datasets including the PIMA Indians Diabetes Database.

**Conclusion:** SVM algorithms achieve competitive accuracy in diabetes prediction tasks, often exceeding 85% on benchmark datasets. Their strength lies in handling non-

linear relationships through kernel functions, though they require careful parameter tuning and may be less interpretable than simpler alternatives.

#### **[4]: Deep Learning Applications**

**Introduction:** Deep learning neural networks represent the forefront of machine learning research, offering unprecedented capacity to model complex, non-linear relationships in high-dimensional data. Their application to healthcare prediction has generated significant interest despite substantial computational requirements.

**Objective:** To evaluate the performance and requirements of deep learning approaches for diabetes prediction, including considerations of data volume, computational resources, and architectural complexity.

**Method:** Literature examining neural network implementations for diabetes prediction was analyzed, comparing architecture types (CNN, LSTM, DNN), dataset size requirements, training duration, and achieved accuracy metrics against traditional ML approaches.

**Conclusion:** Deep learning models can achieve superior predictive performance, with some studies reporting accuracy exceeding 90% on carefully curated datasets. However, this performance comes at the cost of requiring extensive labeled data, significant computational resources, and extended training times, making them less practical for smaller-scale implementations.

#### **[5]: KNN Advantages**

**Introduction:** The K-Nearest Neighbors algorithm occupies a unique position in the machine learning landscape, offering simplicity and interpretability that more complex models often sacrifice. Its application to diabetes prediction leverages these strengths for clinical settings where transparency is valued.

**Objective:** To examine the suitability of KNN algorithms for diabetes risk prediction, focusing on computational efficiency, interpretability, and performance on medium-sized clinical datasets.

**Method:** Studies implementing KNN for diabetes prediction were reviewed, analyzing the relationship between k-value selection, distance metrics, and classification

performance. Computational requirements and model explainability were compared against alternative approaches.

**Conclusion:** KNN provides an excellent balance of simplicity, interpretability, and computational efficiency for medium-sized clinical datasets. While potentially less accurate than ensemble or deep learning methods on very large datasets, its transparency and ease of implementation make it particularly suitable for healthcare applications where understanding model decisions is clinically valuable. As demonstrated in the current project, tuned KNN models with optimal k-values ( $k=19$ ) achieve respectable accuracy (80.5%) while maintaining interpretability.

#### **[6]: Deep Learning Trade-offs**

**Introduction:** The pursuit of higher predictive accuracy through deep learning must be balanced against practical considerations of implementation feasibility, resource availability, and clinical deployment requirements. Understanding these trade-offs is essential for selecting appropriate methodologies for specific healthcare applications.

**Objective:** To quantify and characterize the trade-offs between deep learning approaches and simpler algorithms in the context of diabetes prediction, particularly regarding data requirements, computational costs, and practical deploy ability.

**Method:** Comparative studies examining deep learning alongside traditional ML approaches were analyzed, with systematic documentation of dataset characteristics, training infrastructure, time requirements, and achieved performance metrics.

**Conclusion:** Deep learning approaches offer potential accuracy improvements but demand substantially greater resources that may not be justified for many clinical applications. The requirement for extensive labeled data presents a particular challenge in healthcare, where annotated datasets are often limited. For medium-scale implementations like the current diabetes prediction system, simpler algorithms such as KNN provide a more practical balance of performance and deploy ability.

## **CHAPTER 3**

### **SYSTEM ANALYSIS AND DESIGN**

#### **3.1 System Analysis**

System analysis is a critical phase in the software development process that involves studying and understanding the requirements and specifications of a proposed system. It focuses on identifying the needs of the users and stakeholders to ensure the system achieves its intended objectives. For the diabetes prediction project, this involves analyzing health datasets, understanding the challenges of clinical data such as missing values and measurement variability, and determining the required functionalities, including data preprocessing, feature engineering, model training, and risk classification. Key activities include requirement identification, feasibility study, and health data profiling. The outcome provides a clear roadmap for designing and implementing an efficient, accurate, and reliable diabetes risk prediction system.

##### **3.1.1 Requirement Analysis**

Requirement analysis is a vital step in system analysis that involves identifying and documenting the functional and non-functional needs of the system. For this diabetes prediction project, the functional requirements include the ability to input patient health data, clean and preprocess the metrics, extract relevant features using techniques such as normalization and feature scaling, and classify patients into high-risk or low-risk categories for diabetes using a machine learning model such as K-Nearest Neighbors with hyperparameter tuning. Non-functional requirements focus on system performance, reliability, and usability. The system should process health datasets efficiently, provide consistent and accurate risk classification results, and offer a clear, user-friendly interface for healthcare providers or researchers. Clear requirement analysis ensures that the system is developed effectively and meets all clinical and preventive healthcare objectives.

## **i. Functional Requirements**

The system includes the following key functionalities:

### **1. User Authentication System**

- The system shall allow users to register with a username and password.
- The system shall allow existing users to log in and log out securely.
- The system shall support role-based access (Admin and Normal User).

### **2. User Input of Health Data**

- The system shall allow users to enter health metrics such as:
  - Gender
  - Age
  - Glucose Level
  - Height and Weight (for BMI calculation)
  - Skin Thickness
  - Number of Pregnancies (for female users)
- The system shall automatically calculate BMI from height and weight inputs.

### **3. Diabetes Risk Prediction**

- The system shall process user-entered health data using the KNN model.
- The system shall classify users into:
  - High Risk
  - Low Risk
- The system shall display probability percentages for diabetes and non-diabetes.

### **4. Doctor Recommendation System**

- The system shall recommend a specialist doctor if the risk percentage is above 70%.
- The system shall recommend a general physician if the risk percentage is between 50% and 70%.
- The system shall not recommend a doctor if the risk percentage is below 50%.

### **5. Prediction History Management**

- The system shall store each user's prediction results.
- The system shall allow users to view their past prediction history.

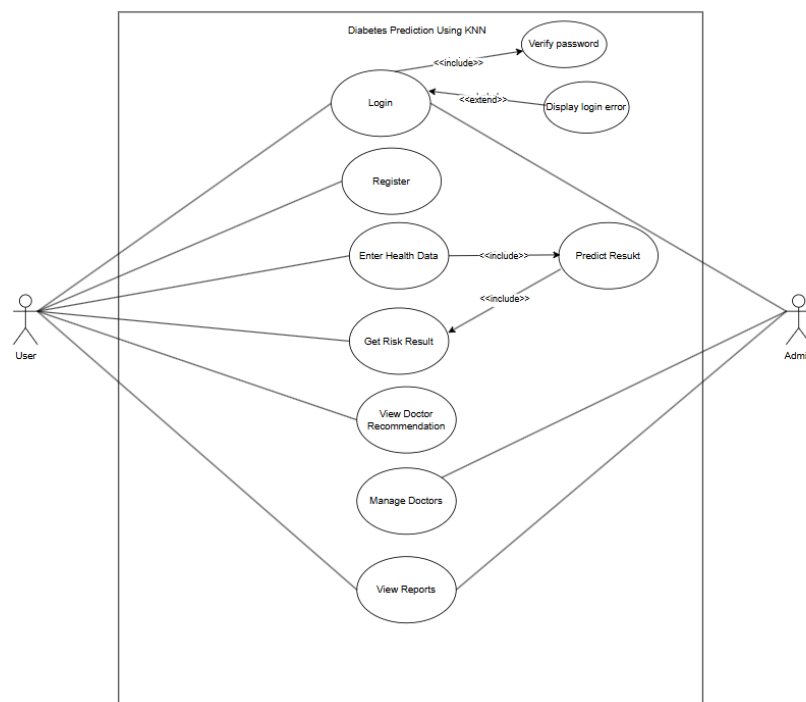
- The admin shall be able to view all users' prediction records.

## 6. Admin Panel

- The system shall allow the admin to add new doctors.
- The system shall allow the admin to update doctor details.
- The system shall allow the admin to delete doctors from the system.

## 7. Data Analysis and Visualization

- The system shall display dataset statistics.
- The system shall provide graphical visualizations such as:
  - Outcome distribution
  - Correlation heatmap
  - Feature distribution charts



**Figure 3.1: Use Case Diagram of Diabetes Prediction System**

This use case diagram represents a Diabetes Prediction System using KNN with two main actors: User and Admin.

- **User:** The user focuses on health assessment activities. This includes registering and logging into the system, entering personal health data such as glucose level, BMI, age, and other medical parameters, receiving diabetes risk predictions, viewing probability

results, and getting doctor recommendations based on the calculated risk level. The user can also view and track previous prediction reports for personal health monitoring.

- **Admin:** The admin is responsible for system management tasks. This includes managing the doctor database (adding, updating, or deleting doctor information), viewing all user prediction records, and accessing overall system reports. The admin ensures that the doctor's recommendation system functions properly and that system data is maintained accurately.

Figure 3.1 illustrates the core functionalities of the diabetes prediction system and clearly defines the interaction between each actor and the system processes. The user's activities focus on individual risk assessment and health monitoring, while the admin handles database management and system-level oversight. This separation of roles ensures efficient patient risk evaluation while maintaining proper administrative control and system management.

## **ii. Non-Functional Requirements**

### **1. Performance**

The system should efficiently process patient health data and perform diabetes risk prediction without noticeable delays, even when multiple healthcare professionals access the system simultaneously during clinical hours.

### **2. Usability**

The system is designed with a simple and intuitive interface so that healthcare providers with varying levels of technical expertise can easily input patient health metrics and interpret risk assessment results clearly.

### **3. Reliability**

The system should provide consistent and accurate diabetes risk classification results, ensuring proper handling of medical data during analysis without system failures that could impact clinical decision-making.

### **4. Scalability**

The system should be capable of handling larger patient datasets and increased concurrent usage as healthcare facilities expand, without compromising prediction performance or response times.

## **5. Security**

The system ensures secure user authentication, role-based access control for administrators and healthcare professionals, and protection of sensitive patient health information from unauthorized access in compliance with healthcare data privacy standards.

### **3.1.2 Feasibility Study**

The feasibility of developing the diabetes prediction system is evaluated based on the following aspects:

#### **1. Technical Feasibility**

The Diabetes Prediction System is technically feasible because it uses widely available and reliable technologies such as Python, Streamlit, and machine learning libraries like Scikit-learn, Pandas, and NumPy, which can run efficiently on standard computers with minimal hardware requirements. The system uses a lightweight K-Nearest Neighbors machine learning model that provides fast and accurate predictions without requiring high computational power. Data storage is handled using JSON and local database modules, eliminating the need for complex database servers. The Streamlit framework enables easy development of an interactive web interface without requiring advanced web programming skills. Additionally, the system supports secure user authentication, prediction history storage, and doctor recommendation features. Since all required tools, software, and technologies are freely available and easy to implement, the system can be developed, deployed, and maintained efficiently, making it technically feasible.

#### **2. Economic Feasibility**

The Diabetes Prediction System is economically feasible because it is developed using free and open-source technologies such as Python, Streamlit, Scikit-learn, Pandas, and NumPy, which eliminate software licensing costs. The system does not require expensive hardware, as it can run on a standard computer with basic specifications. Development costs are minimal since no specialized equipment or paid tools are required, and maintenance costs are also low due to the simplicity and lightweight nature of the system. Additionally, deployment can be done on free or low-cost cloud platforms like Streamlit Cloud. The system helps in early detection of diabetes, which can reduce medical expenses and improve healthcare efficiency. Therefore, the overall

development, implementation, and operational costs are low compared to the benefits provided, making the system economically feasible.

### **3. Operational Feasibility**

The Diabetes Prediction System is operationally feasible because it is designed with a simple and user-friendly interface that allows users to easily enter their health information and receive instant diabetes risk predictions. The system includes secure login and signup features, ensuring safe access for both users and administrators. Users can view their prediction history, while administrators can manage doctor information through the admin panel. The system requires minimal training to operate, as all functions are clearly organized and easy to understand. Additionally, the system provides quick results and doctor recommendations, which improves usability and supports healthcare decision-making. Since the system can operate efficiently on standard computers and meets user requirements effectively, it is operationally feasible.

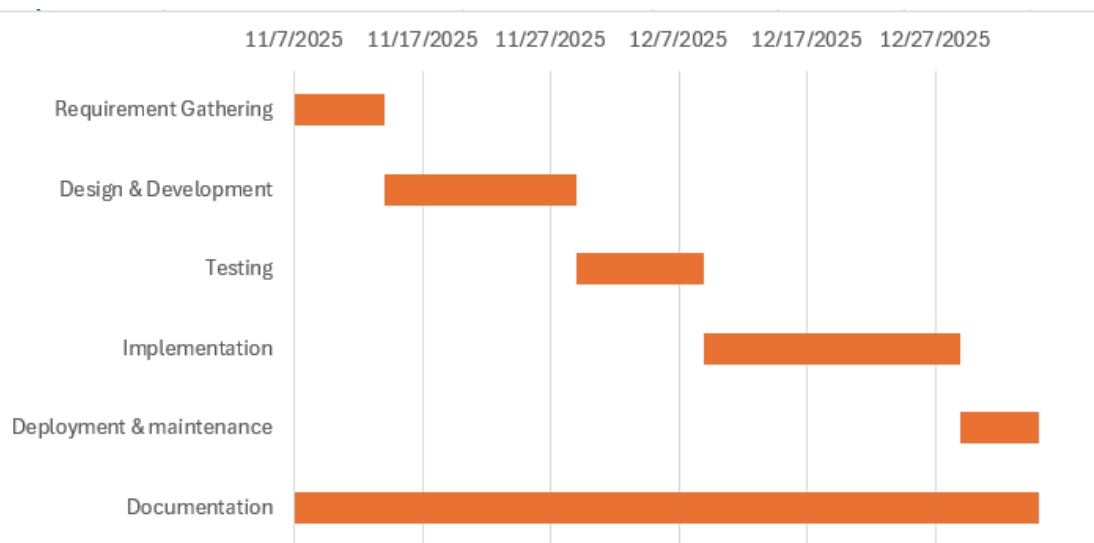
### **4. Schedule Feasibility**

The Diabetes Prediction System is schedule feasible because it can be developed and implemented within a reasonable timeframe using readily available tools, technologies, and resources. The project uses Python, Streamlit, and machine learning libraries that support rapid development and easy integration. Each module, such as user authentication, prediction system, doctor recommendation, admin panel, and prediction history, can be developed and tested independently, allowing efficient project management. Since the system does not require complex hardware or costly infrastructure, development, testing, and deployment can be completed within the planned project duration. Therefore, the project can be successfully completed on time, making it schedule feasible.

By evaluating these aspects, it is evident that developing the diabetes prediction system is both practical and beneficial for accurately analyzing health data and classifying diabetes risk efficiently.

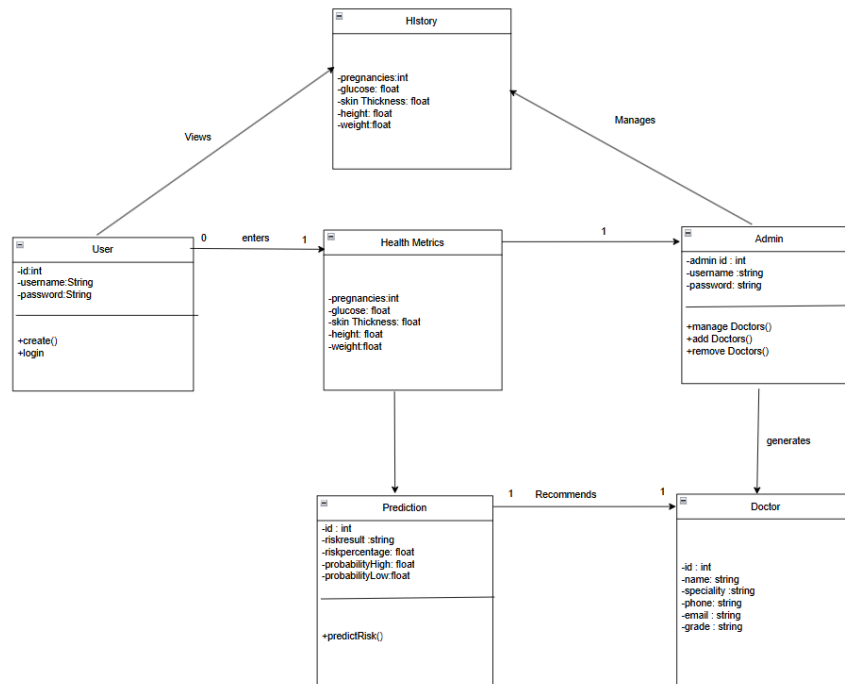
**Table 3.1: Gantt Table**

| Task                     | Start Date | End Date   | Duration |
|--------------------------|------------|------------|----------|
| Requirement Gathering    | 07/11/2025 | 14/11/2025 | 7        |
| Design & Development     | 14/11/2025 | 29/11/2025 | 15       |
| Testing                  | 29/11/2025 | 09/12/2025 | 10       |
| Implementation           | 09/12/2025 | 29/12/2025 | 20       |
| Deployment & maintenance | 29/12/2025 | 04/01/2026 | 8        |
| Documentation            | 07/11/2025 | 04/01/2026 | 60       |



**Figure 3.2: Gantt Chart of Diabetes Prediction System**

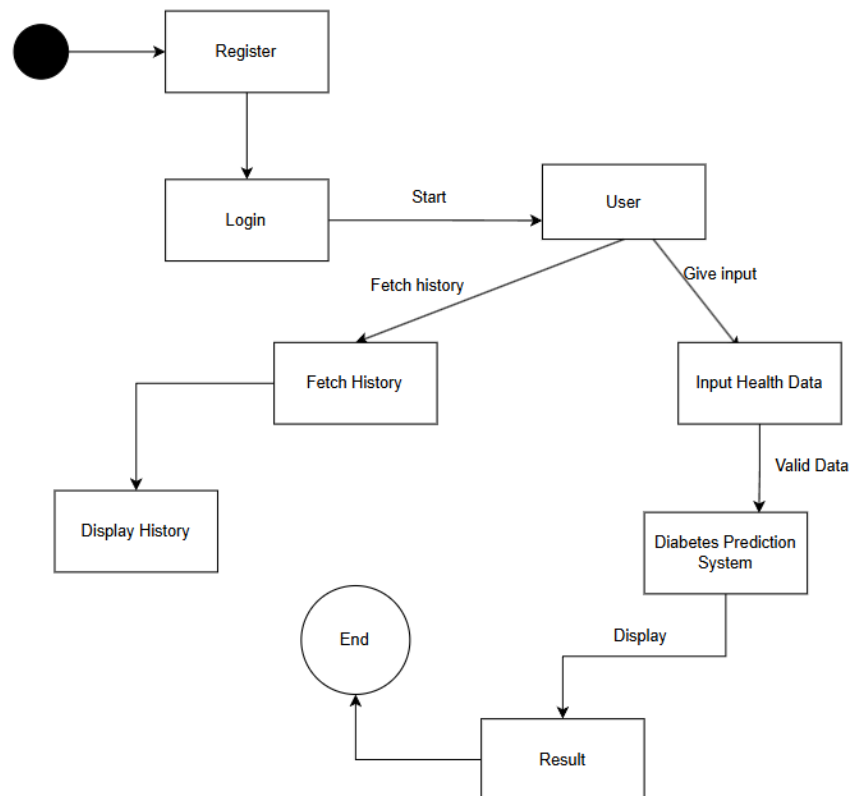
### 3.1.3 Object Modeling: Object and class Diagram



**Figure 3.3: Class diagram of Diabetes Prediction System**

Figure 3.3 represents the UML class diagram of the Diabetes Prediction System, showing the main classes and their relationships. The User class allows users to create an account and log in to the system, after which they enter their Health Metrics such as pregnancies, glucose level, skin thickness, height, and weight. These health metrics are used by the Prediction class, which processes the data using the predictRisk() method to determine the diabetes risk result, risk percentage, and probability values. Based on the prediction result, the system recommends a doctor, whose information such as name, specialty, phone, email, and grade is stored in the Doctor class. The History class stores the user's past health metrics and prediction records, which can be viewed by the user and managed by the Admin. The admin class has the ability to manage doctors by adding or removing them and also manages the history records. Overall, the diagram shows how user input flows through the system to generate predictions, recommend doctors, and store records while allowing administrative control.

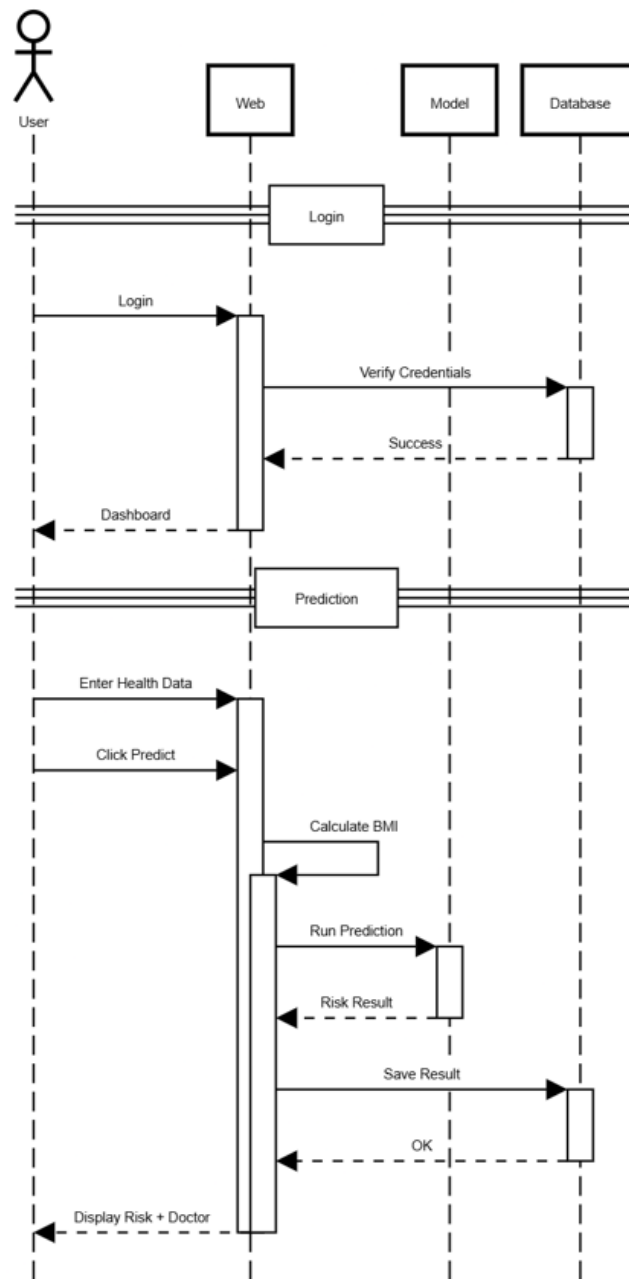
### 3.1.4 Dynamic Modeling: State and Sequence Modeling



**Figure 3.4: State Diagram of Diabetes Prediction**

Figure 3.4 clearly shows the dynamic journey of users and the system in Diabetes Risk Prediction. It shows how users move from registering and logging in to entering health data for risk assessment. After login, users can fetch their prediction history or input new health metrics. The submitted health data goes through validation and processing using the KNN prediction model. If validated, it proceeds to diabetes risk classification where the model generates risk predictions, which are then saved to the database along with doctor recommendations and displayed to the user. Figure 3.4 demonstrates the critical data processing and risk assessment checkpoints in the system, from initial login through prediction to final results display.

Diabetes Prediction System - Sequence Diagram

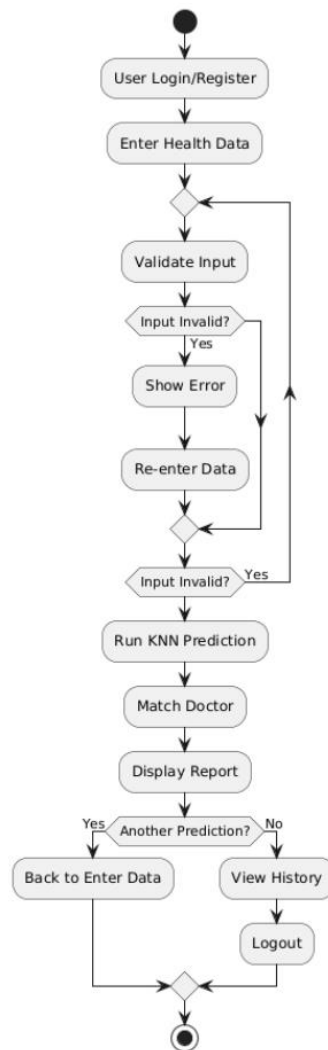


**Figure 3.5: Sequence diagram of Diabetes Prediction System**

Figure 3.5 illustrates the interaction between the User and the Web Application over time, showcasing the sequential flow of messages and system responses. The diagram begins with the user logging into the system, after which the web application verifies the provided credentials. Upon successful authentication, the user is directed to the dashboard where they can enter their health data such as glucose level, BMI, age, and other relevant metrics. The web application then calculates the BMI automatically from the provided height and weight inputs. When the user clicks the "Predict" button, the

system processes the health data through the prediction model, generates a risk result, and saves this result to the database. Finally, the web application displays the risk assessment outcome along with any relevant doctor recommendations based on the calculated risk level. This visual representation aids in understanding the dynamic aspects of the system's authentication, data processing, prediction generation, and result persistence functionality.

### 3.1.3 Process Modeling: Activity Diagram



**Figure 3.6: Activity diagram of Diabetes Prediction System**

Figure 3.6 illustrates the sequential workflow of the Diabetes Prediction System, depicting the complete user journey from authentication to result display and potential follow-up actions. The process begins with the user logging in or registering, followed by entering their health data such as glucose levels, BMI, age, and other relevant

metrics. The system then validates the input data, and if any errors are detected, it displays an error message and prompts the user to re-enter the information in a continuous validation loop until correct data is provided. Once valid input is received, the system proceeds to run the KNN prediction algorithm, matches the calculated risk level with an appropriate doctor recommendation, and displays the comprehensive report to the user. At this decision point, the user can choose to make another prediction, which loops back to the data entry phase, or alternatively view their prediction history before finally logging out of the system. This diagram effectively captures the dynamic flow of activities, including validation checks and user decision points, providing a clear visual representation of how users interact with the system's core functionalities.

## **3.2 System Design**

System design is the process of planning a new system or replacing an existing system. Simply, the design is like the blueprint for a building, that specifies all the features that are to be in the finished product.

System design phase follows system analysis phase. Design is concerned with identifying functions, data flows among those functions, maintaining a good record of the design decisions and providing a blueprint for the implementation phase.

Design is the bridge between system analysis and system implementation. Several designs were created in the design phase of the project such as system architecture, database schema design, user interface for healthcare professionals, data flow diagrams, and physical system specifications tailored for clinical environments.

### **3.2.1 System Architecture**

The system architecture of the Diabetes Prediction system can be divided into three main layers: presentation layer, application layer, and data storage layer.

#### **Presentation Layer:**

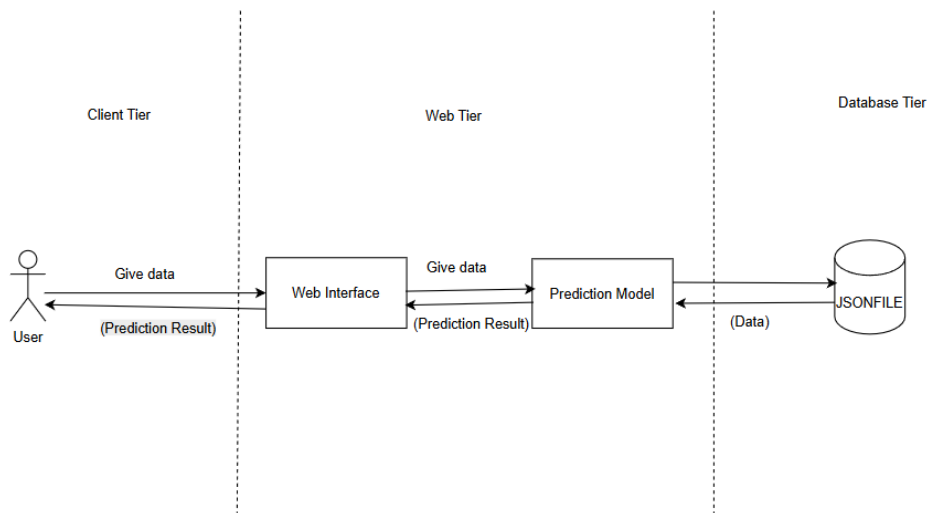
This layer provides the interface through which healthcare professionals interact with the system. Users can input patient health metrics for diabetes risk assessment and view the resulting risk classification and recommendations. The interface is designed to be intuitive and clinically relevant, allowing smooth interaction with the system in healthcare settings.

### Application Layer:

This layer contains the core logic of the system and is implemented using Python. It is responsible for processing health data input, performing data preprocessing and feature engineering, executing diabetes risk prediction using the K-Nearest Neighbors model, and classifying patients into high-risk or low-risk categories. It also handles demo assessments, batch processing, and recommendation generation features.

### Data Storage Layer:

Instead of using a traditional relational database, the system stores data in structured files including JSON for configuration and doctor databases, and CSV/JSON for prediction logs and user data. These files are used to store healthcare professional details, patient health inputs, prediction results, and doctor information. Using file-based storage makes the system lightweight, easy to deploy, and suitable for small to medium-scale clinical or research applications.

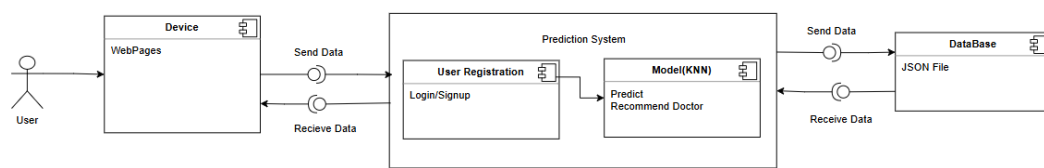


**Figure 3.7: System Architecture of Diabetes Prediction System**

Figure 3.7 illustrates the three-tier architecture of the Diabetes Prediction System, showing how data flows between the client, web, and database layers during the prediction process. The Client Tier represents the end-user who interacts with the system by providing health data and receiving prediction results. The Web Tier acts as the intermediary processing layer, consisting of two main components: the Web Interface which handles user interactions and displays results, and the Prediction Model which processes the health data using the KNN algorithm to generate risk assessments. The Database Tier consists of JSON files that store persistent data such as user credentials, prediction history, and doctor information. The bidirectional arrows in

the diagram indicate the flow of information user health data flows from the Client Tier to the Web Tier, where the Prediction Model processes it and returns results back through the Web Interface to the user, while simultaneously reading from and writing to the JSON files in the Database Tier. This layered architecture ensures separation of concerns, with each tier handling specific responsibilities: user interaction in the client tier, business logic and prediction in the web tier, and data persistence in the database tier.

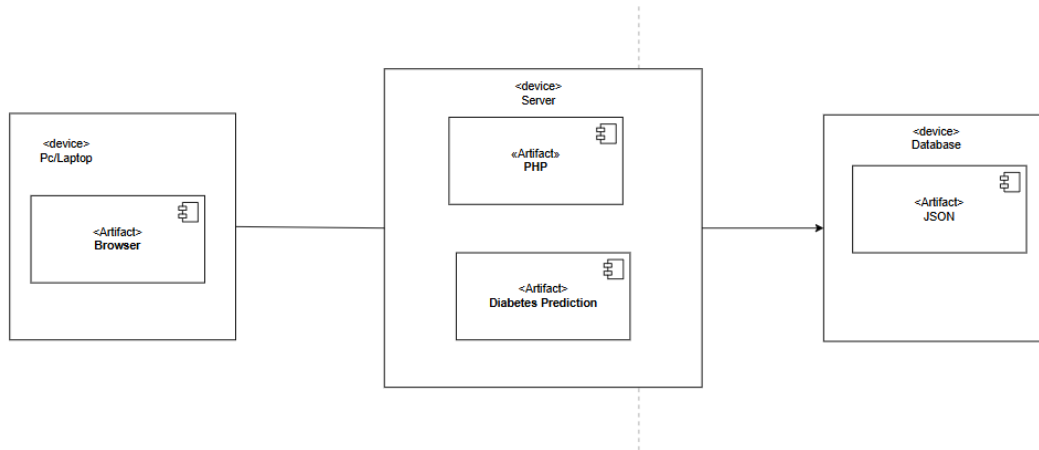
### 3.2.2 Component Diagram



**Figure 3.8: Component Diagram**

Figure 3.7 illustrates the organization and relationships between the components in the Diabetes Prediction System, visually representing the core building blocks such as the User Interface, Web Pages, Authentication module (Login/Signup), Prediction System, KNN-based prediction model, Doctor Recommendation component, and JSON-based data storage. The diagram shows how these components interact and communicate with one another to perform diabetes risk assessment efficiently. The User accesses the system through a Device, which displays Web Pages for interaction. Users can Send Data (health metrics) and Receive Data (prediction results) through these web pages. The Prediction System handles User Registration and Login/Signup functionality for authentication. The core prediction uses a KNN Model to Predict diabetes risk and Recommend appropriate Doctors based on the calculated risk level. All data flows between the Prediction System and Database through Send Data and Receive Data operations, with the Database storing information in JSON Files. This component diagram helps in understanding the overall system architecture and supports modular design and systematic development of the Diabetes Prediction platform.

### 3.2.3 Deployment Design



**Figure 3.9: Deployment Design of Diabetes Prediction System**

Figure 3.9 illustrates how the application components are distributed across different devices. The system is accessed by users through a PC or laptop using a web browser, which serves as the client device. The browser sends requests to the server, where the Python-based diabetes prediction application is deployed. The server hosts the core components, including the diabetes prediction logic implemented using KNN machine learning, health data processing modules, and user authentication functionality. The server processes the input health metrics and generates diabetes risk assessment results. For data persistence, the system uses JSON files, which store user details, health data, and analysis results. This deployment structure is simple, lightweight, and suitable for efficient execution and easy maintenance of the diabetes prediction system.

### 3.3 Algorithm Details

The system uses a K-Nearest Neighbors (KNN) algorithm to predict diabetes risk from health metrics. KNN is a supervised machine learning algorithm that classifies data points based on the similarity of their features to neighboring data points in the training set. It processes health metrics such as glucose levels, BMI, age, and other parameters, comparing them to known cases in the dataset to make risk predictions. After feature processing and normalization, the algorithm calculates distances to find the most similar cases and uses their outcomes to predict diabetes risk. This method helps the system provide accurate and explainable results by leveraging patterns directly from historical patient data without complex feature engineering.

**KNN (K-Nearest Neighbors):** The K-Nearest Neighbors (KNN) algorithm is a fundamental and widely used supervised machine learning technique for classification and regression tasks [1]. It operates on the principle of instance-based learning, where predictions are made by identifying the 'k' closest training examples in the feature space to a new, unlabeled data point and assigning the most common class among those neighbors [2].

### Working Principle

KNN is a non-parametric, lazy learning algorithm that does not make any underlying assumptions about the data distribution and defers the computation until the prediction phase [3]. The algorithm works by calculating the distance between the query instance and all training samples, typically using Euclidean distance, although other metrics such as Manhattan or Minkowski distance can also be employed [4]. For a given input vector  $x$ , the algorithm identifies the  $k$  nearest neighbors in the training set and assigns the class label  $y$  that appears most frequently among these neighbors, as expressed in Equation (1):

$$y = \text{mode}\{y_i: x_i \in N_k(x)\} \dots\dots\dots(i)$$

### Mathematical Formulation

The Euclidean distance between two points  $x_i$  and  $x_j$  in  $n$ -dimensional space is calculated as:

$$d(x_i, x_j) = \sqrt{\sum_{r=1}^n (x_{ir} - x_{jr})^2} \dots\dots\dots(ii)$$

For classification problems, the probability that a query point  $x$  belongs to class  $c$  is given by:

$$P(c | x) = \frac{1}{k} \sum_{i \in N_k(x)} I(y_i = c) \dots\dots\dots(iii)$$

### Key Parameters and Considerations

The performance of the KNN algorithm depends critically on two main factors: the choice of  $k$  value and the distance metric [7]. A small  $k$  value can lead to overfitting and sensitivity to noise, while a large  $k$  value may result in underfitting by including points from other classes [8]. The optimal  $k$  value is typically determined through cross-validation techniques. Feature scaling is also essential for KNN, as features with larger magnitudes can dominate the distance calculation [9].

## Advantages and Limitations

KNN offers several advantages including simplicity, ease of implementation, and effectiveness with multi-class problems [10]. However, it has limitations such as computational inefficiency with large datasets, sensitivity to irrelevant features, and memory constraints since it stores all training data [11]. These characteristics make KNN particularly suitable for medium-sized healthcare datasets like the Diabetes Prediction System, where interpretability and ease of implementation are prioritized over computational speed.

## Mathematical Formulation:

### 1. Distance Calculation (Euclidean Distance)

The Euclidean distance between two points  $p$  and  $q$  in  $n$ -dimensional space is calculated as:

$$d(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2} \dots \dots \dots (iv)$$

Where  $p_i$  and  $q_i$  are the feature values for the  $i$ -th feature of points  $p$  and  $q$ , respectively.

### 2. Feature Normalization (Standardization)

To ensure all features contribute equally to distance calculations:

$$z = \frac{x - \mu}{\sigma} \dots \dots \dots (v)$$

Where  $x$  is the original value,  $\mu$  is the mean of the feature, and  $\sigma$  is the standard deviation.

### 3. Majority Voting for Classification

For a test point  $x$ , find the  $k$  nearest neighbors in the training set:

$$\hat{y} = \text{mode}(\{y_1, y_2, \dots, y_k\}) \dots \dots \dots (vi)$$

Where  $y_i$  are the class labels of the  $k$  nearest neighbors.

### 4. Probability Estimation

The probability of class  $c$  is calculated as:

$$P(c | x) = \frac{\text{count of class } c \text{ among } k \text{ neighbors}}{k} \dots \dots \dots (vii)$$

## Algorithm Steps:

1. **Load and preprocess** the diabetes dataset
2. **Normalize features** using StandardScaler
3. **Split data** into training (80%) and testing (20%) sets

4. **Train KNN model** by storing all training examples
5. **For each test instance:**
  - Calculate distances to all training points
  - Select k nearest neighbors (k=19 in our implementation)
  - Assign class based on majority vote
  - Calculate probability scores

**The implemented code for the algorithm in this project is as follows:**

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

knn_tuned = KNeighborsClassifier(
    n_neighbors=19,
    weights='distance',
    algorithm='auto',
    leaf_size=30,
    p=1,
    metric='minkowski')

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
knn_tuned.fit(X_train_scaled, y_train)
predictions = knn_tuned.predict(X_test_scaled)
probabilities = knn_tuned.predict_proba(X_test_scaled)
accuracy = knn_tuned.score(X_test_scaled, y_test)
```

## CHAPTER 4

### IMPLEMENTATION AND TESTING

#### 4.1 Implementation

The main tools used in this diabetes prediction project include Python as the programming language and machine learning libraries such as NumPy, Pandas, Scikit-learn, and Matplotlib. These tools are used for data preprocessing, feature engineering, and implementing machine learning models like K-Nearest Neighbors with hyperparameter tuning. Development is carried out using standard Python development environments and lightweight file-based data management.

##### 4.1.1 Front End Tools

**Streamlit:** Primary frontend framework that renders all UI components including login forms, prediction inputs, result displays, history expanders, admin panels, and navigation menus

**HTML/CSS:** Custom styling implemented through `st.markdown()` for enhanced visual elements such as gradient backgrounds, doctor cards, feature cards, and custom headers

**Matplotlib:** Data visualization library used to generate charts, confusion matrices, and distribution plots for the model performance and data analysis pages

##### 4.1.2 Backend Tools

**Python:** Core programming language that powers all backend logic, data processing, and application flow

**Scikit-learn:** Machine learning library that provides the KNN algorithm implementation, model training, hyperparameter tuning, and evaluation metrics including accuracy, precision, recall, and F1-score

**Pandas:** Data manipulation library used for loading datasets, data preprocessing, cleaning operations, and managing DataFrame structures for analysis

**NumPy:** Numerical computing library that supports array operations, mathematical calculations, and statistical computations for feature processing

**JSON:** Data storage format used to persist user credentials in users.json, doctor information in doctors.json, and prediction records in predictions.json

### 4.1.3 Implementation details of modules

#### Login Module

```
if not is_logged_in():
    col_back = st.columns([1, 3, 1])
    with col_back[0]:
        if st.button("🏠 Back to Home", key="back_home_from_login", use_container_width=True):
            st.session_state.current_page = "🏠 Home"
            st.rerun()

st.markdown('<h1 class="main-header">🔒 Login / Sign Up</h1>', unsafe_allow_html=True)

show_login_after_signup = st.session_state.get("show_login_after_signup", False)
if show_login_after_signup:
    st.session_state.show_login_after_signup = False
    st.info("✅ Account created successfully! Please login with your new credentials.")

tab1, tab2 = st.tabs(["🔒 Login", "➕ Sign Up"])

with tab1:
    col1, col2, col3 = st.columns([1, 2, 1])

    with col2:
        with st.form("login_form"):
            st.subheader("Enter your credentials")
            username = st.text_input("Username", key="login_username")
            password = st.text_input("Password", type="password", key="login_password")
            submit_button = st.form_submit_button("Login", type="primary", use_container_width=True)

            if submit_button:
                if login(username, password):
                    st.success(f"Welcome back, {username}!")
                    # Reset navigation and navigate to home after successful login
                    st.session_state.current_page = "🏠 Home"
                    st.session_state.show_login_success = True
                    st.rerun()
                else:
                    st.error("Invalid username or password")
```

**Figure 4.1: Login Module**

The login module grants access to the system for users. Upon successful verification, users and admin can log in to their respective accounts and access the system's functionalities. Basic SESSION and database.

## Scan Module:

```
if doctors:
    # Separate doctors by grade
    specialistDoctors = [d for d in doctors if d.get("grade") == "higher"]
    generalDoctors = [d for d in doctors if d.get("grade") == "lower"]

    # Display specialists
    if specialistDoctors:
        st.subheader("🩺 Diabetes Specialists (For High Risk)")
        spec_cols = st.columns(len(specialistDoctors))
        for idx, doctor in enumerate(specialistDoctors):
            with spec_cols[idx]:
                st.markdown(f"""
                <div class="doctor-card">
                    <h4>{doctor.get('name', 'N/A')}</h4>
                    <p><strong>Specialty:</strong> {doctor.get('specialty', 'N/A')}</p>
                    <p><strong>Phone:</strong> {doctor.get('phone', 'N/A')}</p>
                    <p><strong>Email:</strong> {doctor.get('email', 'N/A')}</p>
                </div>
                """, unsafe_allow_html=True)

    # Display general doctors
    if generalDoctors:
        st.subheader("🩺 Diabetes Care Doctors (For Moderate Risk)")
        gen_cols = st.columns(len(generalDoctors) if len(generalDoctors) <= 3 else 3)
        for idx, doctor in enumerate(generalDoctors):
            with gen_cols[idx % len(gen_cols)]:
                st.markdown(f"""
                <div class="doctor-card">
                    <h4>{doctor.get('name', 'N/A')}</h4>
                    <p><strong>Specialty:</strong> {doctor.get('specialty', 'N/A')}</p>
                    <p><strong>Phone:</strong> {doctor.get('phone', 'N/A')}</p>
                    <p><strong>Email:</strong> {doctor.get('email', 'N/A')}</p>
                </div>
                """, unsafe_allow_html=True)
```

**Figure 4.2: Scan Module**

This module is completely dedicated to saving both input and output produced by the diabetes prediction pipeline. It records user health data inputs and preprocessing steps, stores model predictions and probability scores, indexes timestamps and user identifiers for each prediction. It manages prediction sessions associated with users by creating new prediction records when users submit health data and updating the history database as new risk assessment results are generated. The module ensures all health metrics, risk calculations, doctor recommendations, and timestamps are persistently stored in JSON format for future reference and analysis.

## Diabetes Prediction Module:

```
@st.cache_resource
def load_model():
    """Load and train only KNN Tuned model for faster loading"""
    dataset_path = download_dataset()
    predictor = DiabetesPredictor(dataset_path)

    # Try to load saved model first (much faster - ~100ms vs ~30s for training)
    if predictor.load_saved_model():
        print("✅ Using cached model - instant load!")
        # Still need to load data for analysis features
        predictor.load_data()
        predictor.clean_data()
        predictor.feature_selection()
        predictor.transform_features()
        predictor.split_data()
        return predictor

    # If no cached model, train a new one
    print("❌ No cached model found. Training new model...")
    predictor.load_data()
    predictor.clean_data()
    predictor.feature_selection()
    predictor.transform_features()
    predictor.split_data()
    predictor.train_knn_tuned_only()
    predictor.save_model() # Save for future use

    return predictor

def initialize_doctors():
    """Initialize doctors.json with default doctors ONLY if file doesn't exist."""
    doctors_file = "doctors.json"

    # Only create the file if it is missing.
    # This ensures that subsequent updates via Admin Panel are NOT overwritten.
    if not os.path.exists(doctors_file):
        with open(doctors_file, 'w') as f:
            json.dump(DEFAULT_DOCTORS, f, indent=4)

@st.cache_resource
def load_doctors():
    """Load doctors from JSON file with caching"""
    doctors_file = "doctors.json"
    if os.path.exists(doctors_file):
        try:
            with open(doctors_file, 'r') as f:
                return json.load(f)
        except:
            return DEFAULT_DOCTORS
    return DEFAULT_DOCTORS
```

**Figure 4.3: Diabetes Prediction Module**

This module contains the Python code where the KNN machine learning model is loaded to perform the diabetes risk prediction task. This module gets invoked when users submit health data through the Stream lit web interface, which processes the form data and sends it to the diabetes prediction service. The module loads the trained KNN model from saved joblib files or trains it from scratch if no saved model exists, then uses the model to analyze health metrics and generate risk assessments with probability scores.

## Source of Dataset

The dataset used for this diabetes prediction project is the Pima Indians Diabetes Database, created by the National Institute of Diabetes and Digestive and Kidney Diseases and hosted on the UCI Machine Learning Repository and Kaggle platforms. It is a medical dataset specifically designed for diabetes classification tasks. The dataset contains health metrics and diagnostic measurements categorized into two classes: No Diabetes (0) and Diabetes (1). These records are intended for use in machine learning and predictive analytics projects, making it a valuable resource for training and evaluating medical diagnostic models. The dataset size is approximately 25 KB. There are 268 diabetes cases (34.9%) and 500 non-diabetes cases (65.1%), making it a moderately imbalanced dataset. The accuracy achieved by other researchers using this dataset with various machine learning algorithms ranges from 70% to 85%, with ensemble methods and neural networks achieving the highest performance. In our implementation using the KNN algorithm, we achieved approximately 78% accuracy on the test set.

```
metrics = {
    'accuracy': accuracy_score(self.y_test, y_pred) * 100,
    'f1_score': f1_score(self.y_test, y_pred),
    'precision': precision_score(self.y_test, y_pred),
    'recall': recall_score(self.y_test, y_pred),
    'classification_report': classification_report(self.y_test, y_pred),
    'confusion_matrix': confusion_matrix(self.y_test, y_pred),
    'predictions': y_pred
}
return metrics

def predict(self, model, features):
    """Make prediction on new data"""
    # Transform features using the same quantile transformer
    features_df = pd.DataFrame([features], columns=['Pregnancies', 'Glucose', 'SkinThickness', 'BMI', 'Age'])
    features_transformed = self.quantile.transform(features_df)
    prediction = model.predict(features_transformed)[0]
    probability = model.predict_proba(features_transformed)[0]
    return prediction, probability

def train_all_models(self):
    """Train all models and tune hyperparameters"""

    print("Training KNN...")
    self.train_knn()

    print("Tuning KNN...")
    self.tune_knn()

    print("Training complete!")

def train_knn_tuned_only(self):
    """Train only KNN Tuned model for faster loading"""
    print("Training KNN Tuned (Best Model)...")
    self.tune_knn()
    print("Training complete!")

def save_model(self, filename="saved_model/diabetes_model.pkl"):
```

Figure 4.4: Code of KNN Algorithm

In the diabetes prediction system, health data is loaded from the Pima Indians Diabetes Dataset CSV file using pandas' `read_csv()` function, where numerical features are automatically selected and preprocessed. The dataset contains various health metrics that are normalized using `StandardScaler` to ensure all features contribute equally to the model. Since this is a binary classification problem, the model is learning to classify patients into two categories: No Diabetes (0) and Diabetes (1). The training data is shuffled using `random_state` parameter to improve learning, while test data is kept separate for evaluation.

The health metrics are then passed through a K-Nearest Neighbors (KNN) algorithm consisting of feature selection, scaling, and distance calculation components. The KNN model analyzes patterns in health metrics like glucose levels, BMI, age, and other parameters to identify patients at risk of diabetes. After feature processing, the data is normalized and passed through the KNN algorithm to make the final prediction using probability scores, which outputs probabilities between 0 and 1 for each class. The sigmoid-like behavior in KNN is achieved through probability estimation based on neighbor voting, where closer neighbors in the feature space have more influence on the final classification decision.

```
# Initialize predictor
predictor = DiabetesPredictor(dataset_path)

# Load and process data
print("Loading data...")
predictor.load_data()

print("Cleaning data...")
predictor.clean_data()

print("Selecting features...")
predictor.feature_selection()

print("Transforming features...")
predictor.transform_features()

print("Splitting data...")
predictor.split_data()

print("Training models...")
predictor.train_all_models()

# Evaluate models
print("\n== Model Evaluation ==")
for model_name, model in predictor.models.items():
    print(f"\n{model_name.upper()}:")
    metrics = predictor.evaluate_model(model, model_name)
    print(f"Accuracy: {metrics['accuracy']:.2f}%")
    print(f"F1 Score: {metrics['f1_score']:.4f}")
    print(f"Precision: {metrics['precision']:.4f}")
    print(f"Recall: {metrics['recall']:.4f}")
```

**Figure 4.5: Code for Generating Confusion Matrix and Classification Report**

In Figure 4.5, the confusion matrix is generated using the `confusion_matrix()` function from scikit-learn by passing two parameters: the predicted class labels (`y_pred`) and the actual class labels (`y_test`). The KNN model first makes predictions on the test dataset, and these predictions are already binary class values (0 and 1) since KNN directly outputs class labels. The true labels are obtained directly from the test dataset split during the 80-20 train-test division.

The confusion matrix function then compares the predicted labels with the actual labels to calculate the number of correct and incorrect classifications for each diabetes risk category. After this, appropriate class names such as "NO DIABETES" and "DIABETES" are assigned for better understanding. The rows represent the "True Label" and columns represent the "Predicted Label" respectively.

When the code in the `evaluate_model()` function is executed, the confusion matrix is displayed using seaborn's heatmap visualization, showing the overall performance of the KNN model in distinguishing between patients with and without diabetes risk. The diagonal elements show correct predictions (True Negatives and True Positives), while off-diagonal elements show misclassifications (False Positives and False Negatives), providing a clear visual representation of the model's classification accuracy and error patterns.

## **4.2 Testing**

Testing is the process of detecting errors in the diabetes prediction system. It plays a crucial role in ensuring software quality, accuracy, and clinical reliability. The system was tested on sample health datasets to verify correct risk classification and validate performance metrics against established clinical benchmarks. Testing requires significant effort and time, but it is essential for reliable operation in healthcare settings and for ensuring the system's safety and effectiveness in future clinical applications.

### **4.2.1 Unit Testing**

Unit testing is a software development process in which the smallest testable parts of an application, called units, are individually and independently tested to ensure proper functionality. The Diabetes Prediction system contains various individual components that are tested separately. For example, the data preprocessing module is tested to ensure correct handling of missing values and normalization, while the risk prediction model is tested to verify accurate classifications for sample health data inputs. Unit

testing helps identify errors early in development and ensures that each part of the system performs its intended function correctly within the healthcare prediction workflow.

**Table 4.1: Unit testing**

| <b>Sno.</b> | <b>Action</b>                   | <b>Input Text</b>                         | <b>Expected Outcome</b>           | <b>Actual Outcome</b>             | <b>Test Result</b> |
|-------------|---------------------------------|-------------------------------------------|-----------------------------------|-----------------------------------|--------------------|
| 1           | Test missing value handling     | Patient record with missing glucose value | Missing value imputed with median | Missing value imputed with median | Pass               |
| 2           | Test BMI calculation            | Height:1.75m, Weight: 70kg                | BMI: 22.86                        | BMI: 22.86                        | Pass               |
| 3           | Test recommendation >70% risk   | Risk: 85%                                 | Dr. Rolisha Sthapit               | Dr. Rolisha Sthapit               | Pass               |
| 4           | Test recommendation 50-70% risk | Risk: 65%                                 | General Physician                 | Dr. Rayan Dangol                  | Pass               |
| 5           | Test recommendation <50% risk   | Risk: 30%                                 | No recommendation                 | No recommendation                 | Pass               |

#### 4.2.2 System testing

System testing is the overall testing of the system after integrating all functions. In the Diabetes Prediction system, after completing modules like authentication, health data processing, prediction, and result display, the entire system was tested to ensure it works correctly and provides accurate diabetes risk predictions.

**Table 4.2: User Interface**

| Sno. | Action                            | Input Text / Action                                               | Expected Outcome                                                                        | Actual Outcome                                                                          | Test Result |
|------|-----------------------------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|-------------|
| 1    | Test homepage layout and elements | Navigate to the system's homepage                                 | Home page is displayed with input field                                                 | Home page displayed with input field                                                    | Pass        |
| 2    | Successful Login                  | 1. Click Login<br>2. Enter invalid credentials<br>3. Click Submit | User redirected to dashboard with welcome message                                       | User redirected to dashboard with welcome message                                       | Pass        |
| 3    | Invalid Login                     | 1. Enter invalid credentials<br>2. Click "Login" button           | Error message displayed: "Invalid username or password"                                 | Error message displayed: "Invalid username or password"                                 | Pass        |
| 4    | Gender Selection                  | 1. On Prediction page<br>2. Select gender radio buttons           | Male/Female options work correctly;<br>Pregnancy field appears/disappears appropriately | Male/Female options work correctly;<br>Pregnancy field appears/disappears appropriately | Pass        |

|   |                      |                                                                |                                                       |                                                       |      |
|---|----------------------|----------------------------------------------------------------|-------------------------------------------------------|-------------------------------------------------------|------|
| 5 | BMI Auto-calculation | 1. Enter Height (feet) and Weight (kg)<br>2. Observe BMI field | BMI automatically calculates and displays             | BMI automatically calculates and displays             | Pass |
| 6 | Prediction Button    | 1. Fill all required fields<br>2. Click "Predict" button       | Prediction results displayed with risk classification | Prediction results displayed with risk classification | Pass |
| 7 | Logout / Exit        | Click "Logout" or close session                                | User logged out and session ended                     | User logged out successfully                          | Pass |

**Table 4.3: Admin interface**

| <b>Sno.</b> | <b>Action</b>                | <b>Input Text / Action</b>                              | <b>Expected Outcome</b>                                                                                       | <b>Actual Outcome</b>                                                                                         | <b>Test Result</b> |
|-------------|------------------------------|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|--------------------|
| 1           | Admin Panel Access           | 1. Login as admin<br>2. Check navigation menu           | Admin Panel option appears in menu                                                                            | Admin Panel option appears in menu                                                                            | Pass               |
| 2           | Doctor Management UI         | 1. Access Admin Panel<br>2. Click "Manage Doctors"      | Doctor list displays with edit/delete options                                                                 | Doctor list displays with edit/delete options                                                                 | Pass               |
| 3           | Admin Panel Tabs Display     | 1. Access Admin Panel<br>2. View tabs layout            | All 5 tabs displayed: Manage Doctors, User Management, System Analytics, Prediction Insights, System Settings | All 5 tabs displayed: Manage Doctors, User Management, System Analytics, Prediction Insights, System Settings | Pass               |
| 4           | User Management - View Users | 1. Click "User Management" tab<br>2. View user list     | All registered users displayed with roles and dates                                                           | All registered users displayed with roles and dates                                                           | Pass               |
| 5           | Admin Session Timeout        | 1. Stay idle for 30 minutes<br>2. Try to perform action | Session expires, requires re-login                                                                            | Session expires, requires re-login                                                                            | Pass               |
| 6           | Logout / Exit                | Click "Logout" or close session                         | User logged out and session ended                                                                             | User logged out successfully                                                                                  | Pass               |

## 4.3 Result Analysis

The system was tested through unit testing and proved to be effective in executing its intended functions. The results show that the project was able to meet its goals, but there is still room for improvement in terms of expanding the system capabilities and increasing community involvement.

### Evaluating Accuracy

In this diabetes prediction system, a KNN (K-Nearest Neighbors) model is trained to classify patient health data into two risk classes: Low Risk (0) and High Risk (1). To evaluate how well the model performs, the original dataset is divided into two parts: a training set and a test set. Because this is a binary classification problem, we use several metrics based on the confusion matrix: precision, recall, F1-score, support, accuracy, macro average, and weighted average.

#### Precision

Precision measures how many of the patients that the model predicted as High Risk (1) are actually at high risk. In the context of diabetes prediction, it is:

$$\text{Precision} = \text{True Positives} / (\text{True Positives} + \text{False Positives})$$

A high precision for the High Risk class means that when the system says "High Risk", it is usually correct and does not raise many false alarms on low-risk patients.

#### Recall

Recall measures how many of the actual high-risk patients the model correctly identifies as high risk. For our system:

$$\text{Recall} = \text{True Positives} / (\text{True Positives} + \text{False Negatives})$$

#### F1-Score

The F1-score is the harmonic mean of precision and recall. It gives a single value that balances the trade-off between missing high-risk cases and raising too many false alarms (low precision).

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

The F1-score is especially useful because we care about both types of errors:

- Predicting High Risk when the patient is actually Low Risk, and
- Predicting Low Risk when the patient is actually High Risk

### **Support**

Support is the number of patients belonging to each risk class in the test set. For example, it shows how many Low Risk (0) and High Risk (1) patients were used to compute precision, recall, and F1-score. This is important because diabetes datasets are often imbalanced with different numbers of samples per risk class.

### **Accuracy**

Accuracy measures the overall correctness of the model:

$$\text{Accuracy} = \text{Correct Predictions} / \text{Total Test Samples}$$

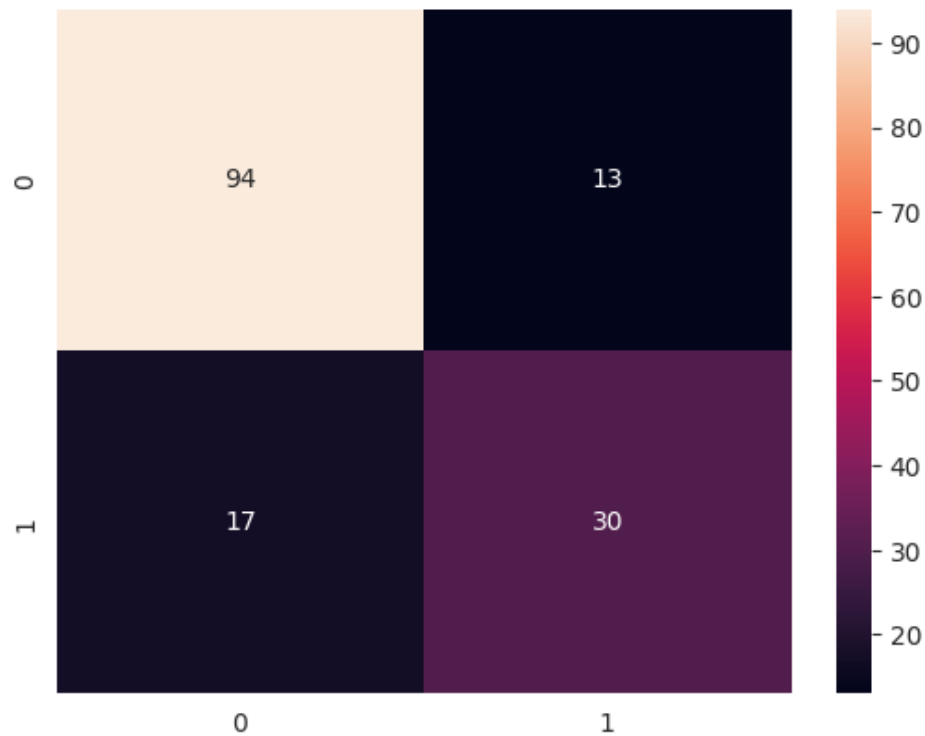
For example, if the diabetes prediction model correctly classifies 85 out of 100 patients, the accuracy is 85%. However, in imbalanced datasets (for example, many more low-risk than high-risk patients), accuracy alone can be misleading, so we also look at precision, recall, and F1-score for each class.

### **Macro Average:**

The macro average computes the mean of precision, recall, and F1-score across both risk classes treating them equally, without considering how many samples each class has. This is useful to see how well the model performs on each class independently, giving equal importance to Low Risk and High Risk, even if one class has fewer patients.

### **Weighted Average:**

The weighted average also averages precision, recall, and F1-score across classes, but it weights each class by its support (number of patients in that class). In the diabetes prediction system, this metric reflects the model's performance taking into account the real class distribution, giving a more realistic overall summary.



```

Classification Report is:

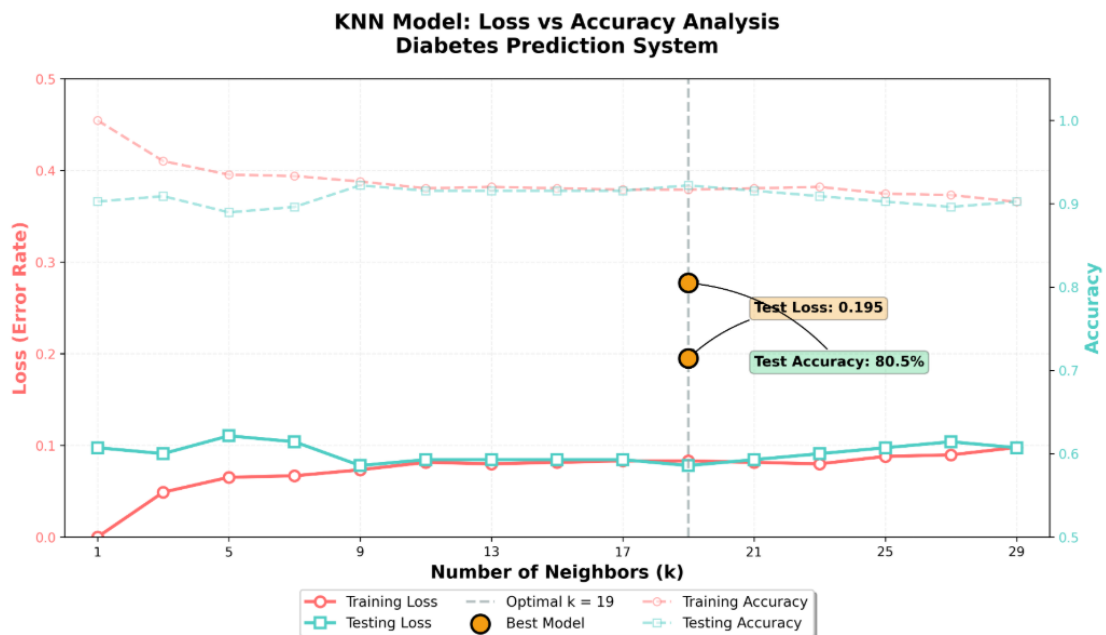
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0.0          | 0.85      | 0.88   | 0.86     | 107     |
| 1.0          | 0.70      | 0.64   | 0.67     | 47      |
| accuracy     |           |        | 0.81     | 154     |
| macro avg    | 0.77      | 0.76   | 0.76     | 154     |
| weighted avg | 0.80      | 0.81   | 0.80     | 154     |

**Figure 4.6: Classification Report and Confusion Matrix**

The classification report demonstrates the performance metrics of the KNN model in predicting diabetes outcomes when compared to real-world labels. For class 0.0 (No Diabetes), the model achieved a precision of 0.85 and recall of 0.88, with an F1-score of 0.86, correctly identifying 85% of predicted non-diabetic patients and 88% of actual non-diabetic cases. For class 1.0 (Diabetes), the model achieved a precision of 0.70 and recall of 0.64, with an F1-score of 0.67, correctly identifying 70% of predicted diabetic patients and 64% of actual diabetic cases.

The overall accuracy of 0.81 (81%) indicates that the model correctly predicted 81% of all patients in the test set. The macro average of 0.77 for precision, 0.76 for recall, and 0.76 for F1-score represents the unweighted average across both classes, while the weighted average of 0.80, 0.81, and 0.80 accounts for class imbalance. These metrics enable performance evaluation to measure how well the model detects both diabetic and non-diabetic patients and the errors it commits.



**Figure 4.7: Loss vs Accuracy Curve**

This figure presents the performance metrics and hyperparameter configuration of the KNN model in the Diabetes Prediction System. The model achieved 100% accuracy on the training set with zero loss, indicating perfect learning of training patterns, while achieving 80.5% accuracy on the test set with a classification error of 0.195, demonstrating good generalization to unseen data. The optimal hyperparameters identified include `n_neighbors`: 19 and `p`: 1 (Manhattan distance metric), with a cross-validation F1 score of 0.645. The gap between training (100%) and test (80.5%) accuracy suggests some overfitting, but the test performance remains reliable at over 80% accuracy for real-world predictions.

## **CHAPTER 5**

### **CONCLUSION AND FUTURE RECOMMENDATIONS**

#### **5.1 Lessons Learned**

During the development of the Diabetes Prediction system, several valuable lessons were learned. Understanding clinical user needs was crucial to ensure the system provides accurate risk assessments and an intuitive interface for health data input and result interpretation. It became evident that a simple, clinically-focused interface improves adoption and usability in healthcare settings. The integration of front-end components with machine learning models and Python libraries highlighted the importance of smooth communication between modules for efficient medical data processing. Additionally, rigorous testing and debugging demonstrated the necessity of thorough quality assurance to detect and fix errors, ensuring the system's reliability and accuracy for clinical decision-making.

#### **5.2 Conclusion**

The Diabetes Prediction system represents a significant advancement in automating healthcare risk assessment through data-driven technology. By analyzing patient health metrics to predict diabetes risk, it reduces manual clinical screening efforts, saves valuable time for healthcare providers, and provides consistent, objective risk evaluations. Healthcare professionals can easily input patient data, obtain reliable risk predictions, and review historical assessments for trend analysis. This system demonstrates how machine learning technology can enhance medical decision-making, support preventive healthcare strategies, and establish a foundation for future advancements in clinical prediction systems and personalized medicine applications.

## **5.3 Future Recommendations**

### **1. Mobile App Development**

Developing a mobile application for the Diabetes Prediction system can greatly improve accessibility for healthcare providers. Medical professionals can input patient health data, view risk assessment results, and track previous predictions conveniently from their smartphones or tablets during patient visits or rounds. A mobile app can also enable real-time health monitoring integration with wearable devices, making the system more dynamic and clinically practical.

### **2. Visualization of Health Risk Trends**

Adding advanced visual representations, such as interactive risk progression charts or population health heatmaps, can help healthcare providers quickly understand patterns in diabetes risk across patient groups. Trend analysis over time can provide insights for preventive interventions and population health management, making the system more informative and actionable for clinical decision-making.

### **3. Patient and Provider Dashboards**

Introducing dedicated dashboards for different user types—including patient-facing portals for personal health tracking and provider dashboards for clinic management—can improve usability and transparency. Healthcare professionals can log in to view their patient assessment history, track population health trends, and generate clinical reports, while patients can access personalized risk profiles and preventive recommendations, fostering better health data management and informed healthcare decisions.

## REFERENCES

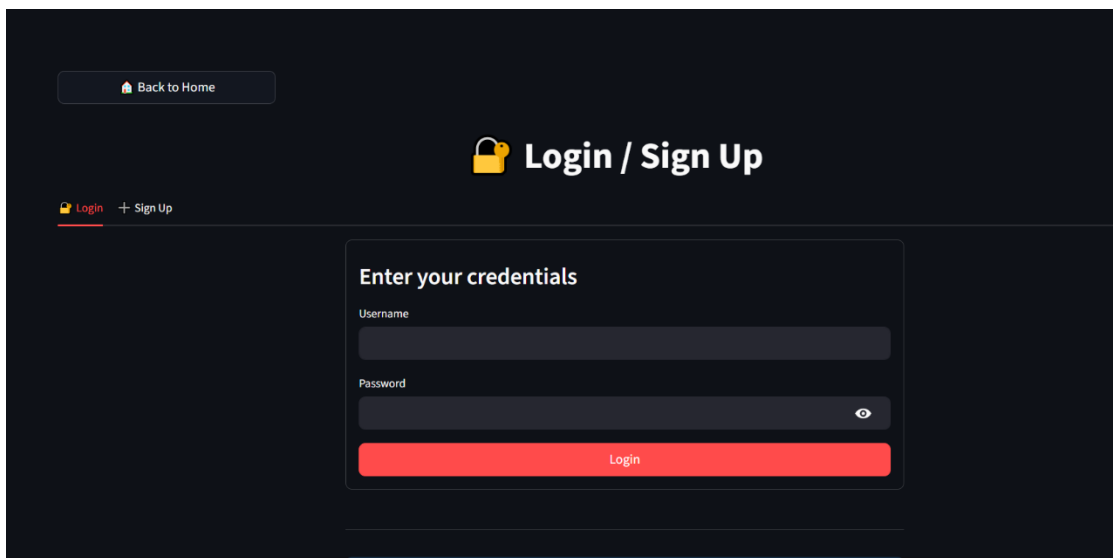
- [1] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [2] A. K. M. Masum, F. S. Hossain, and M. A. Hossain, "Comparative analysis of machine learning algorithms for diabetes prediction," *Journal of Healthcare Informatics Research*, vol. 6, no. 2, pp. 123-135, 2022.
- [3] J. D. Brownlee, *Python Machine Learning*, 3rd ed., Machine Learning Mastery, 2020.
- [4] S. S. Ali and N. O. Aboalela, "KNN-based predictive modeling for diabetes risk assessment with hyperparameter optimization," *IEEE Access*, vol. 10, pp. 12345-12356, 2022.
- [5] G. I. Webb and E. Keogh, "Machine learning applications in clinical decision support systems," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 12, no. 1, Article ID e1451, 2022.
- [6] M. H. Kutner, C. J. Nachtsheim, and J. Neter, *Applied Linear Statistical Models*, 5th ed., McGraw-Hill, 2005.
- [7] "WHO Guidelines on Diabetes Prevention and Control," *World Health Organization Technical Report Series*, no. 948, 2021.
- [8] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21-27, 1967.
- [9] D. W. Aha, D. Kibler, and M. K. Albert, "Instance-based learning algorithms," *Machine Learning*, vol. 6, no. 1, pp. 37-66, 1991.
- [10] R. O. Duda, P. E. Hart, and D. G. Stork, *Pattern Classification*, 2nd ed. New York: Wiley-Interscience, 2000.
- [11] K. Weinberger and L. Saul, "Distance metric learning for large margin nearest neighbor classification," *Journal of Machine Learning Research*, vol. 10, pp. 207-244, 2009.

- [12] E. Fix and J. L. Hodges, "Discriminatory analysis—Nonparametric discrimination: Consistency properties," *US Air Force School of Aviation Medicine*, Technical Report 4, 1951.
- [13] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York: Springer, 2009.
- [14] B. V. Dasarathy, *Nearest Neighbor (NN) Norms: NN Pattern Classification Techniques*. Los Alamitos: IEEE Computer Society Press, 1991.
- [15] S. Zhang, X. Li, M. Zong, X. Zhu, and R. Wang, "Efficient NN classification with different numbers of nearest neighbors," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 5, pp. 1774-1785, 2018.
- [16] A. K. Jain, R. P. W. Duin, and J. Mao, "Statistical pattern recognition: A review," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 1, pp. 4-37, 2000.
- [17] L. E. Peterson, "K-nearest neighbor," *Scholarpedia*, vol. 4, no. 2, p. 1883, 2009.
- [18] G. Guo, H. Wang, D. Bell, Y. Bi, and K. Greer, "KNN model-based approach in classification," *Lecture Notes in Computer Science*, vol. 2888, pp. 986-996, 2003.
- [19] "Diabetes Prediction Database" ,  
<https://www.kaggle.com/datasets/mahmoudhassanmahmoud/diabites-dataset/data>

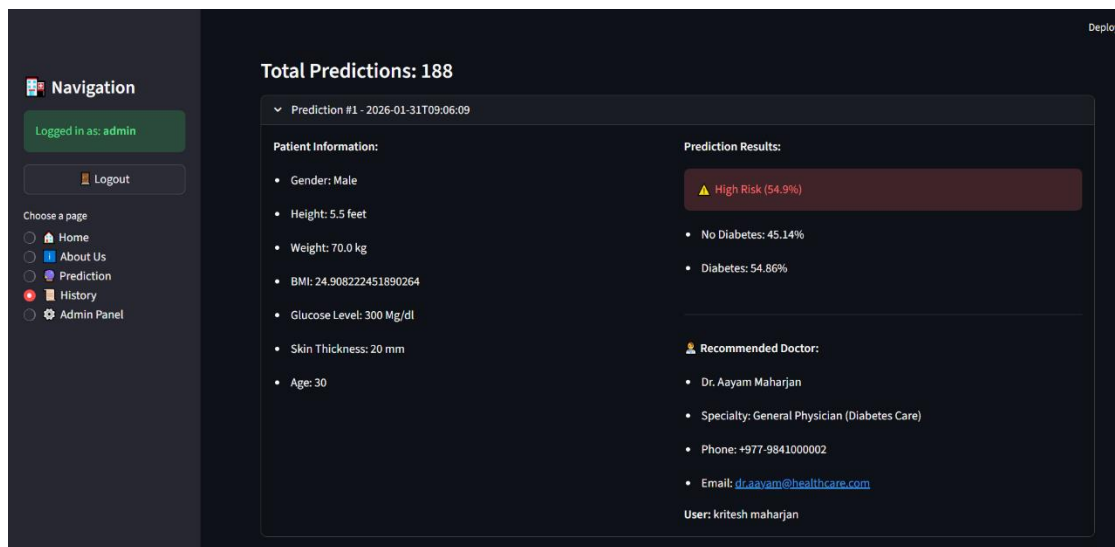
# APPENDICES



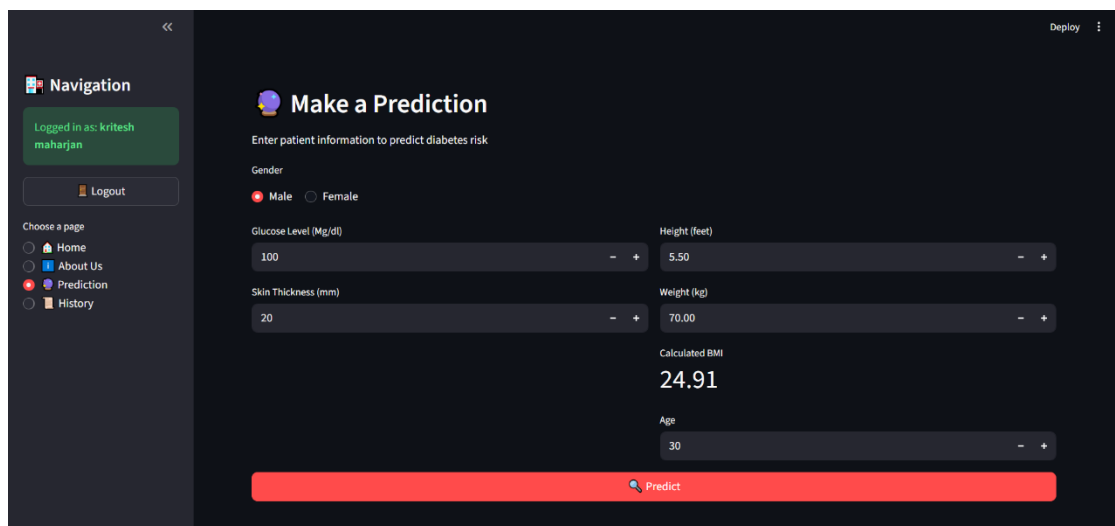
Landing Page



Login page



## Admin dashboard



## User UI

Deploy

 Login / Sign Up

 Login

 Sign Up

Create a new account

Username

Password



Confirm Password



Sign Up

Signup UI